# Inside Figma’s Engineering Culture

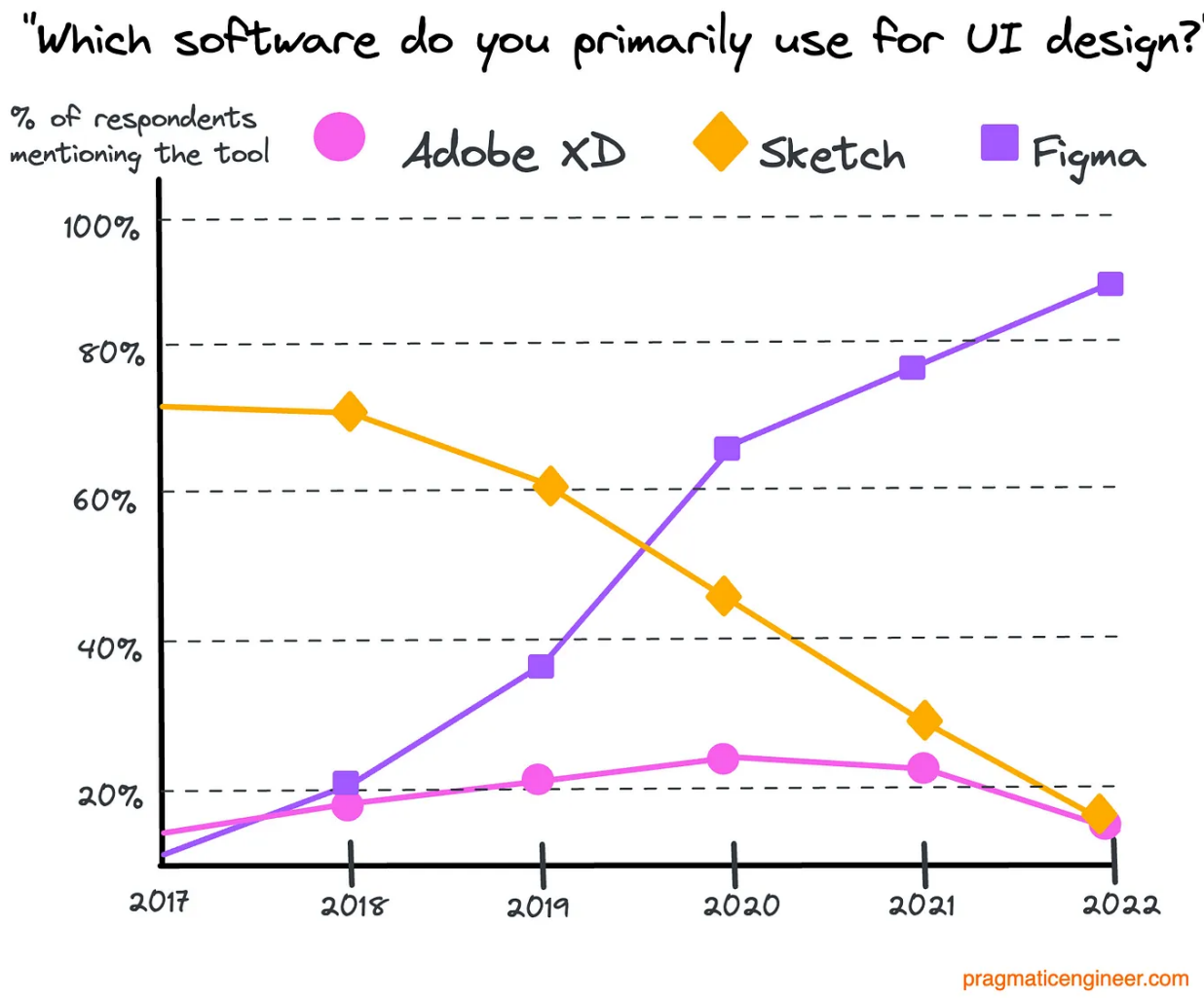
A deep dive into how the dev team works at Figma and their “build with deliberation, build with pride” engineering culture. As told by Figma’s CTO, Kris Rasmussen.

GERGELY OROSZ

APR 4, 2023 • PAID



Figma is a design platform for teams to build products. Founded in 2012, the company launched its first paid product in 2016, which exploded in popularity for Figma to become the market-leading designer collaboration tool it is today.




*Market share of UI design tools, based on the annual UX Design Tools Survey, to which 4,000 designers contributed in 2022*

In September 2022, Adobe acquired Figma for $20B, in what was Adobe's biggest acquisition to date. At the time of publishing, Figma employs more than 1,000 people, and about 300 of these work in tech.

Clearly, Figma has done plenty of things right to become the go-to tool for designers, out-competing companies including Sketch and Adobe. I was interested to learn how the company operates from an engineering perspective, and what inspiration other tech companies could take from Figma. So I reached out to CTO Kris Rasmussen, who was generous in sharing his insights during a conversation which lasted more than two hours.

In this issue, we dig into the engineering culture at Figma, as Kris walks us through these topics:

1. **The engineering team's evolution** and what's unique about Figma's culture.
2. **Engineering structure:** pillars, platform teams, EM/engineer ratios.
3. **Getting things done.** Exploration vs execution, the planning process and prioritizing fixing bugs.
4. **The engineering culture.** Dogfooding, experimentation, quality weeks, onboarding engineers and internal mobility.
5. **The engineering career ladder.** When and how it was built, and its levels.
6. **The tech stack.** React, TypeScript and C++ on the frontend; Ruby, Go and TypeScript on the backend.
7. **Engineering challenges.** Client-side performance, distributed databases and scale.

Tech can be a small world, and in 2016 when at Uber I worked on a major app rewrite with Yuhki Yamashita, who's Figma's Chief Product Officer. He shares details on how Figma builds products in **Lenny's Newsletter** , which is a nice accompaniment to this issue from a product point of view.

*With that, it's over to CTO Kris, who shares details on the engineering side of things at Figma. What follows are his words.*

# 1. The engineering team's evolution

During its early days in 2012-2016, Figma was a very small team working on a very ambitious minimum viable product (MVP,) at the peak of the Web 2.0 movement. Back then, the conventional wisdom was to measure success by whether you could launch something within six months: any longer was too slow.

But the reality is that if you're ambitious about what you're trying to bring to the Web, then it just takes more time to build a quality app, as Figma has proved by ignoring conventional wisdom.

Figma started to ramp up hiring around 2016, when the MVP was ready and looked promising, and the company had 12 engineers. That same year, I started as a contractor, joining full-time in 2017 when the eng team had 15 members.

When I joined, engineering was a single team with one engineering manager. We decided to split the team into two functional areas:

1. Team blockers: focused on the differentiators and collaboration story.
2. Individual blockers: focused on filling in the long tail of feature gaps that blocked product designers from persuading teams to switch tools.

We stuck with this structure until we felt enough blockers were filled. Then the team grew to be large enough that we needed to split it again.

**We organized teams around focus areas** by examining what the most urgent challenges were, and how to temporarily organize teams to resolve them as fast as possible. We were intentionally not seeking a long-term structure for the organization. We ensured everyone understood it was a temporary and fluid approach.

These temporary structures stayed in operation until around 2019 when the team was big enough to start thinking about creating higher-level structures to promote greater continuity and stable ownership over time.

**As a company gets larger and more mature, you need to generate a sense of longer-term ownership.** We came to this conclusion as the engineering team grew. In some cases, engineers want to feel ownership and responsibility for shepherding forward some part of the product or technology. Another driver was to define ownership in the longer-term, as there are often many areas in need of continuous improvement and iteration, on a longer time horizon.

One thing Figma has done well is to stay very detail-oriented. We do not always just focus on the next big thing, but also on the small details that really make a design tool fluid, efficient and something people love to use. Engineers want to feel more ownership, and the product also needs more love in specific areas.

## Unique things about Figma's engineering culture

**Lift your team.** In the early days of Figma, we created the value: 'Lift your team.' This stood out to me when I first got involved with Figma and I observed how supportive and focused people were on uplifting colleagues, not competing with each other.

I think that's carried through in the culture in the sense people don't try to win arguments and similar individualistic behaviors. It's really about the right solution as a group and being very open to feedback. An example of this in action is that our design team has a process called "design crits."

**Design crits.** A unique thing about our design team is that they're very welcoming of engineers, who often join design crits to share and hear feedback. It helps make the early exploration phase of projects more collaborative, versus designing something and then handing it to engineering.

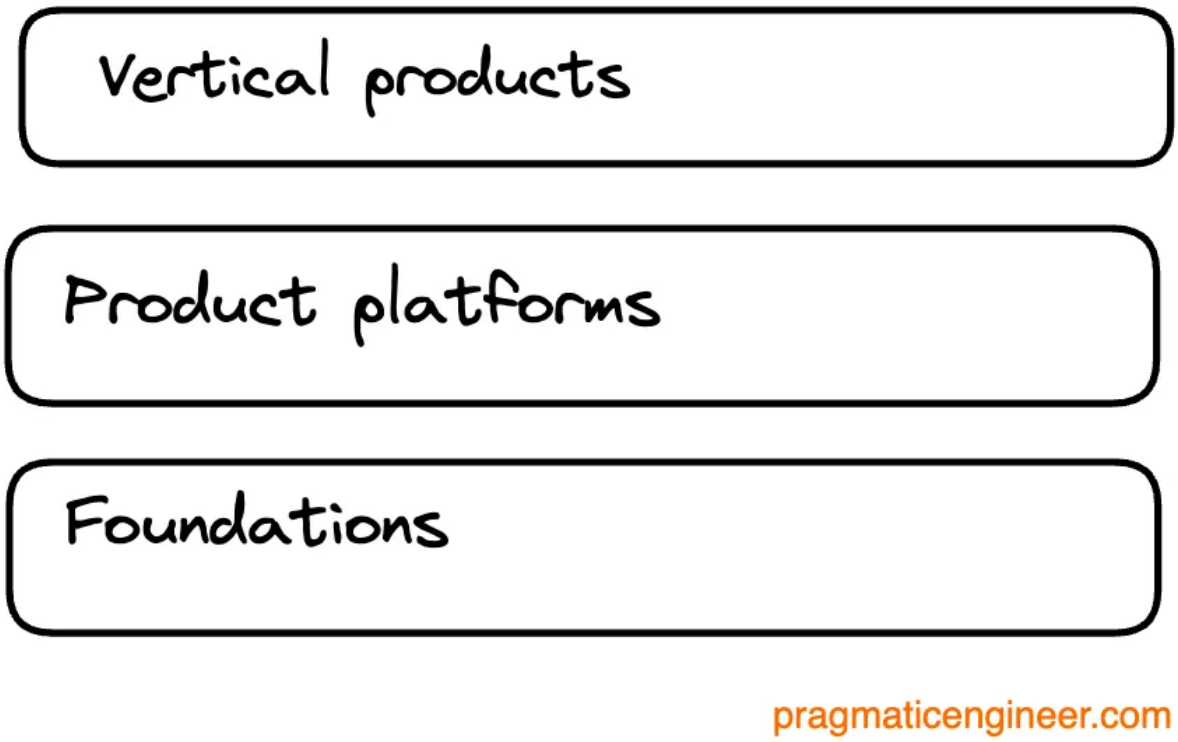
**Close to customers.** We encourage engineers to be very close to customers. While we do have a user research function, engineers still frequently observe user research studies. They engage with customers in private beta channels in Slack, and we try to work closely with customers.

To give an extreme example, both Dylan – the CEO and cofounder of Figma – and I engage with people in support and answer queries on Twitter. I still occasionally jump onto Zoom calls to debug customer issues, to ensure I know what the problems are.

# 2. Engineering structure

## Engineering pillars

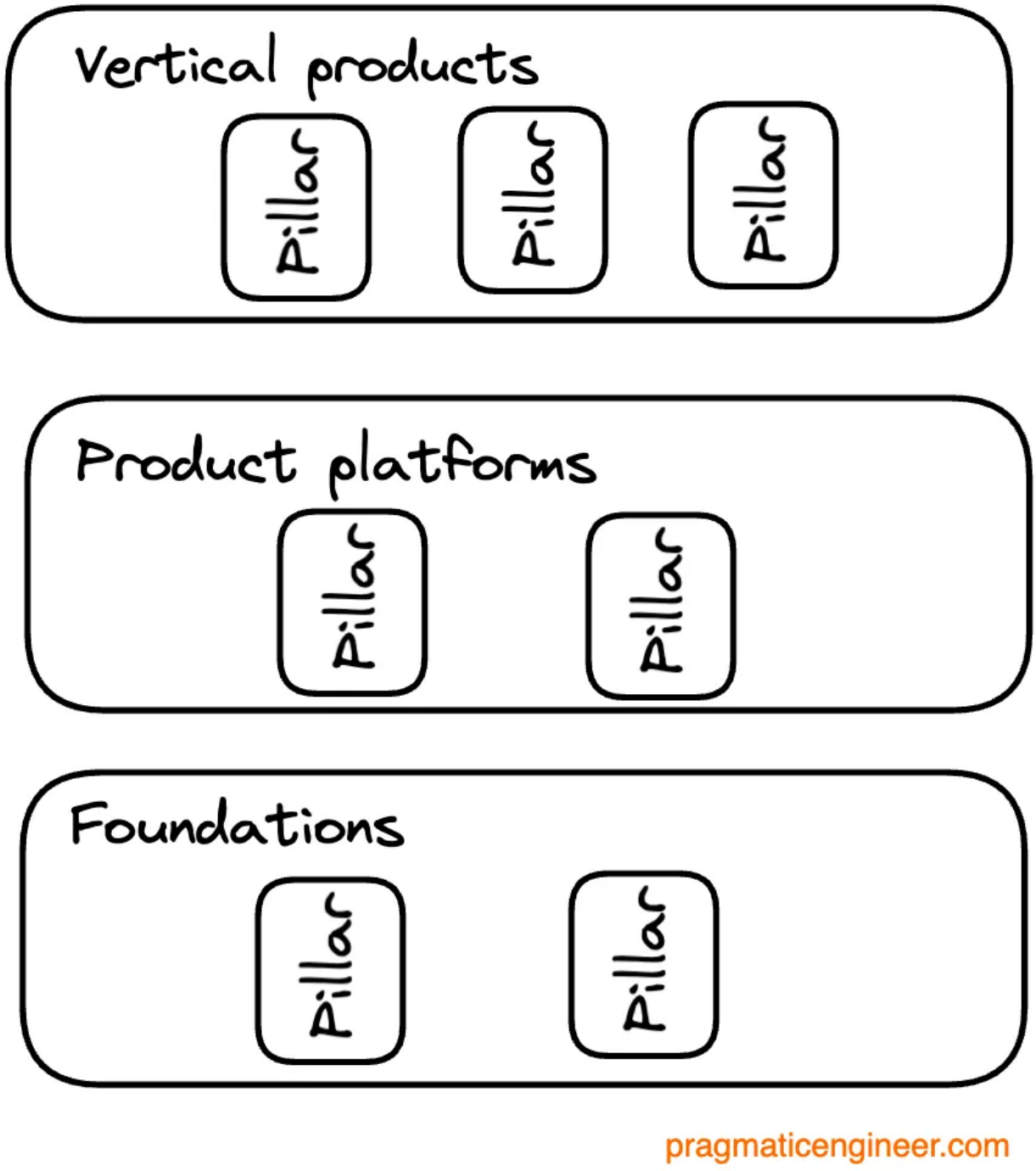
In the early days, we wanted people to work full stack and own things top to bottom. As we grew, we divided the organization into a "sandwich structure."




The 'sandwich structure' of the engineering organization at Figma

**Figma currently has seven engineering pillars.** A pillar is like a product group or a tech group. At the top level, there's pillars focused on vertical products, still thinking about end-user needs. At the mid-level, we're thinking about common features of those products. At the lower level, we're thinking about the shared systems and frameworks which enable everything above.

We try to model different functions around this higher-level pillar structure. Engineering usually is a bit ahead in terms of headcount, compared to functions like product. The management structure tracks the pillar structure almost one-to-one, if not exactly. Product and engineering is also organized according to the pillars.




Pillars within the engineering organization. Pillars refer to groups with shared purpose, and they tend to be either within vertical product, a product platform or a foundational group

Two examples of pillars:

- **The Figma Editor**. This is for the Figma design project and is a vertical pillar.

- **The Creation Engine**. This pillar is an infrastructure team, but focused more on the UI engine side of Figma, versus traditional web scale infrastructure.

What about specialized cases like site reliability engineer (SRE) folks, data team, ML engineers, etc? The way we handle these is similar to smaller functions. At the highest level, we're trying to decide whether or not a group is actually a different function, with a different career ladder, different career path, versus the same function. If it's more of a distinct function or different domain with more specialization, then we're more willing to consider it a pillar of its own.

For example, we currently have a pillar called "Native," a group with domain experts around iOS, Android and Electron. Engineers report into the Native pillar and also work across other pillars. They embed themselves and pair with teams, reporting through a different structure. That's the trade-off we're always making: most people report through the pillar structure, but certain specializations and functions also report outside of it.

## Platform teams

Earlier in the company's history, we had some smaller, platform-like teams. However, when we introduced our second product, FigJam, it became more obvious that the two products – Figma Design and FigJam – shared technology stacks. So it was helpful to create product-agnostic teams that can really focus on common themes and challenges across both products, and these are our platform teams.

Date-driven initiatives were another reason why we created platform teams. Such initiatives frequently swallowed up platform work for a team. So by carving out dedicated teams for important platform work, we could be more intentional about how much time we spent on platform work.

We just call these teams by their name, and don't use special platform names. We try to keep our platform teams working very closely with product managers (PM) and product developers, so they're close to the end user. Even our platform teams stay close to end users which is something many people appreciate about Figma, and is something we want to keep.

## Engineering manager-to-engineer ratios

Our EM-to-engineer ratio is around 1 to 8-10. How did we get to this ratio? It comes down to whether or not an engineering manager has the ability to adequately support their team. Engineering managers also need the bandwidth to work on things their team does to help the company be successful.

When the engineer-to-manager ratio gets too high, for example, more than 10 engineers to each manager, then you're either compromising a manager's ability to support the team, or reducing their ability to think strategically about what their team can do, and what they can do for the company. Our current engineer-to-manager ratio seems like a happy balance.

Not every engineering team has a PM counterpart. For our more infrastructure-focused teams such as for web infrastructure, the PM / engineer ratio is lower. In a sense, engineers wear the PM hat on those teams.

## Upper limit on headcount growth

We have been intentional about growing the workforce. In spite of how quickly the business is growing and how many problems we have, we said we'll never more than double in size during a year. We set that limit because we think faster growth would compromise the culture.

In practice, we've rarely gotten close to doubling the team within a year. As we've scaled up, headcount growth has been just enough to keep up with overall growth and the challenges of the business.

# 3. Getting things done

Projects are very collaborative. From the outside, it'd be natural to assume Product does one thing and Engineering another. But we both are very welcoming for input and we give each other ideas and feedback. On the Product side, Yuhki – Chief Product Officer at Figma – and I work very closely together. Engineers also tend to get involved early in the planning process.

We staff teams so they are full stack and can work across code boundaries.

## Exploration vs execution

We separate exploration projects from execution ones, which helps us be realistic about whether or not a project is ready to be staffed in earnest for execution projects. If a project is more explorative, it shouldn't occupy 100% of a team's or person's time.

Even with the split, we are intentional about involving all functions early during projects, be they exploration or execution ones. Our goal is for people to work collaboratively on figuring out the right problem and the right way to solve it.

In practice, this split is:

1. A project starts in an exploration phase
2. Later, it transitions to a "ready to be executed" state

Of course, iterations still happen during the execution phase, but this phase is concrete enough that we can staff it with more engineers.

## Planning

The technical planning process works alongside the product process. We do Product Requirement Documents for lighter-weight things and Product Reviews for more complex projects. We sometimes break a Product Review down into Problem Alignment and Solution Alignment phases.

While product planning is happening, we present engineering concepts and engineering designs, and share design docs early and often – which is something I push for a lot. We encourage sharing works in progress in Slack, and have certain channels for this.

**Seek out feedback early and often**: this is one of our key values as an engineering organization. Some companies prefer engineering design docs to be polished artifacts and to follow a formal process. We don't do this.

We prefer engineers to write down what they're thinking as they think it, so they can get feedback. We encourage engineers to get feedback from people with strong opinions or prior experience, to do this as early as possible and to let it evolve over time. In this way, our engineering process is less formal.

Depending on the nature of the project, we sometimes run a more formal planning process. For example, if we're talking about modifying a backend system with a lot of load which is critical to stability, we follow more defined steps.

**Engineering pillars differ in terms of what level of formalism they expect for different categories of project.** We want to give each team a little bit of flexibility, and aim to spread the spirit of planning across all teams. Basically, everyone should:

- Document their decisions
- Work in the open
- Welcome feedback

Out of these three, the most important is getting feedback as soon as possible, and then to iterate in the open.

## Project management

We're not overly prescriptive about how teams do project management. We have best practices which we encourage them to follow.

**Using milestones is one common ask of teams;** that every project be broken down into a series of milestones which the teams communicate early. The idea is to encourage everyone to create a culture of accountability so they can trust and rely on each other, versus being overly prescriptive about exactly how they do their work. Our use of milestones is one practice that's stood the test of time.

## Fixing bugs vs building new things

Fixing low-priority bugs within the features you build is something we do. When you're doing feature work, there's a much stronger service level agreement (SLA) around fixing even relatively low-priority bugs. If you notice a low severity bug which you

introduced during the course of your feature work, then it's a high-priority fix because “now” is the right time to fix it. We don't want things to just accumulate over time, and it's much easier to fix them straight away when you have context, than later down the road.

At the same time, if something gets found three months after we launch the product, then it's really up to the team to prioritize it, relative to the overall impact they're trying to deliver.

### There is no single “right” way to do things

Early in my career I jumped around more than many people in my position do. Thanks to this, I got exposure to a lot of different companies and the contrasts between Asana and Airbnb’s early days. In the end, both companies became successful in spite of being different.

Seeing very different paths to success made me realize there's no one right way to do things. It's really about agreeing upon the core principles and values. Sometimes, you need to do more clarifying as things get a little messier and more complex. However, in the end, you need to let people decide the right way to execute, while keeping the goals of the company in mind.

I've been very intentional about not trying to be overly prescriptive around drawing on past experience, because I've learned there's just many different ways to do things.

## 4. Figma’s engineering culture

### Engineering values

Figma has codified four engineering values:

1. Communicate early and often
2. Lift your team
3. Craftsmanship
4. Prioritize impact

We've already discussed values #1 and #2. Craftsmanship is about thoughtfulness and taking care in the work we do. It means being deliberate about what we build and how possible it will be to maintain and extend in the future. This reflects our drive to be thoughtful and sustainable in how we build.

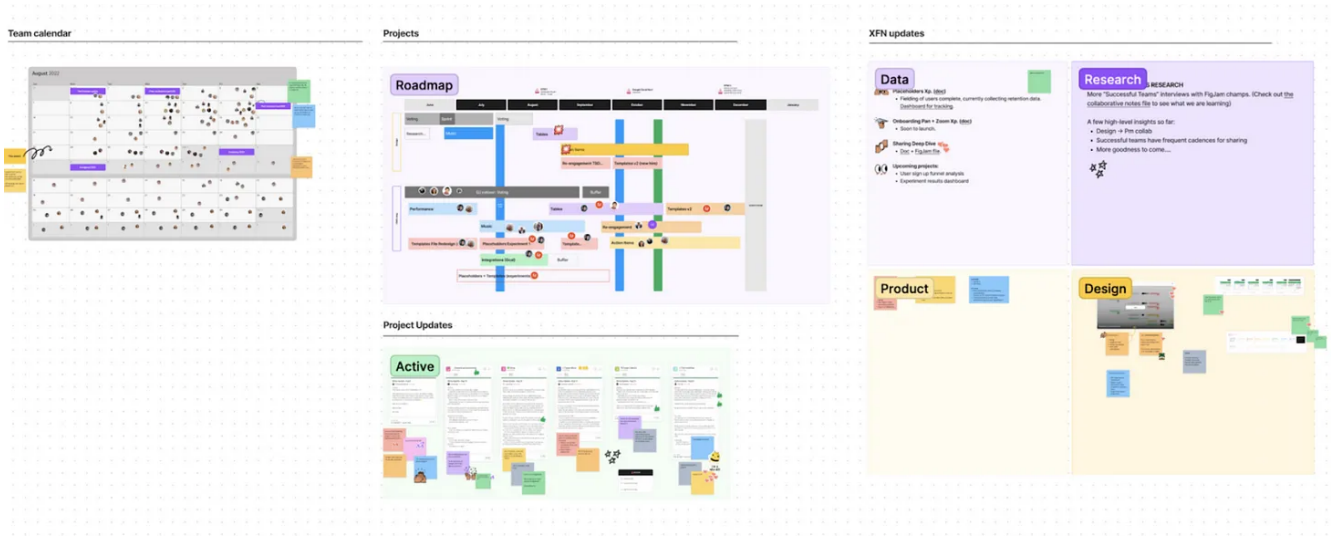
Prioritizing impact doesn't just inform how we pick between projects, but also how we approach them. It ensures that we don't over-engineer in pursuit of sustainability.

We cover the thinking behind each value in this engineering blog post.

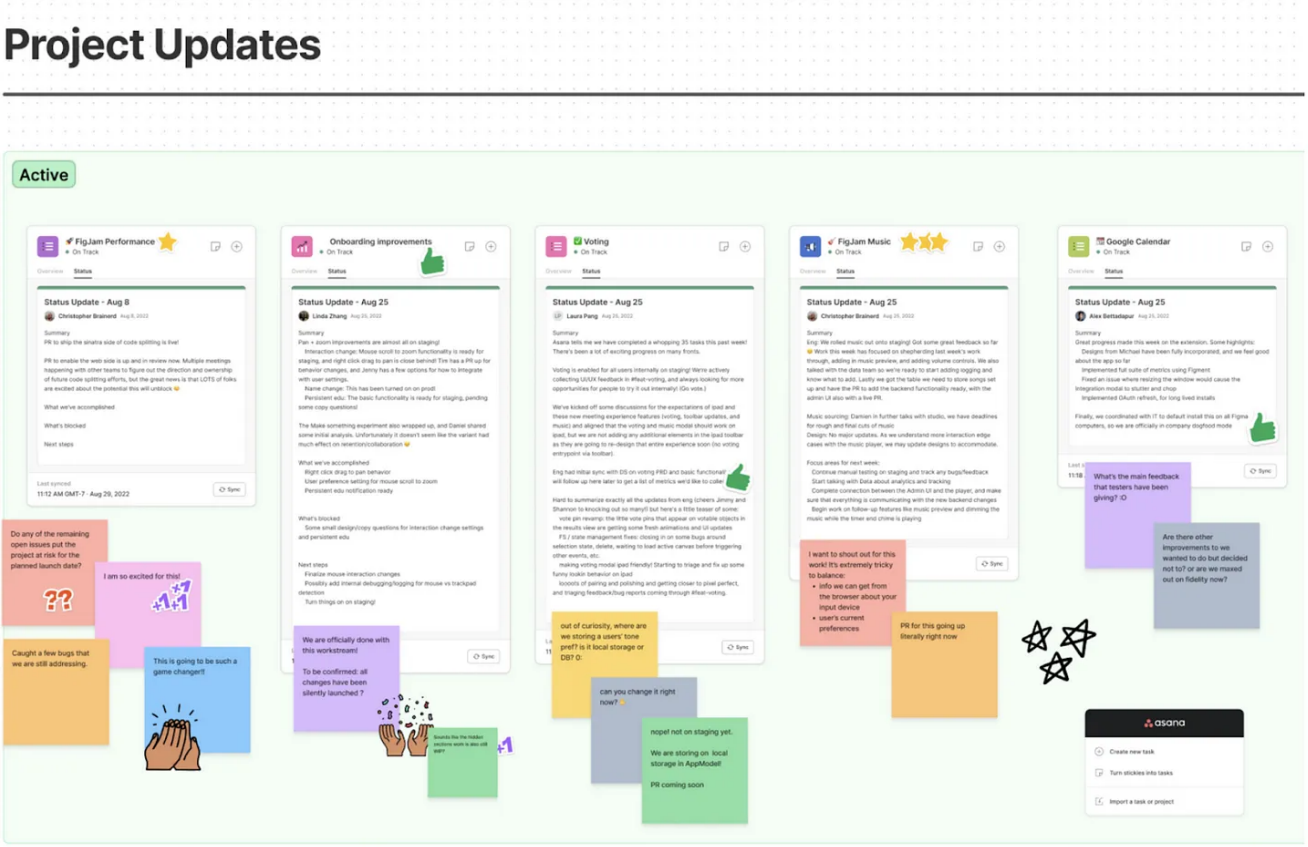
## Dogfooding

It's only natural that we dogfood our products. It's important to note we don't believe we're *only* building for design teams. We are building Figma for *entire* product teams. For example, we introduced FigJam – an online collaborative whiteboard – because we need our own lightweight ways to run meetings, especially in the hybrid world, to get real-time feedback, to make meetings more inclusive, and also leverage the visual capabilities of Figma in a more accessible way.

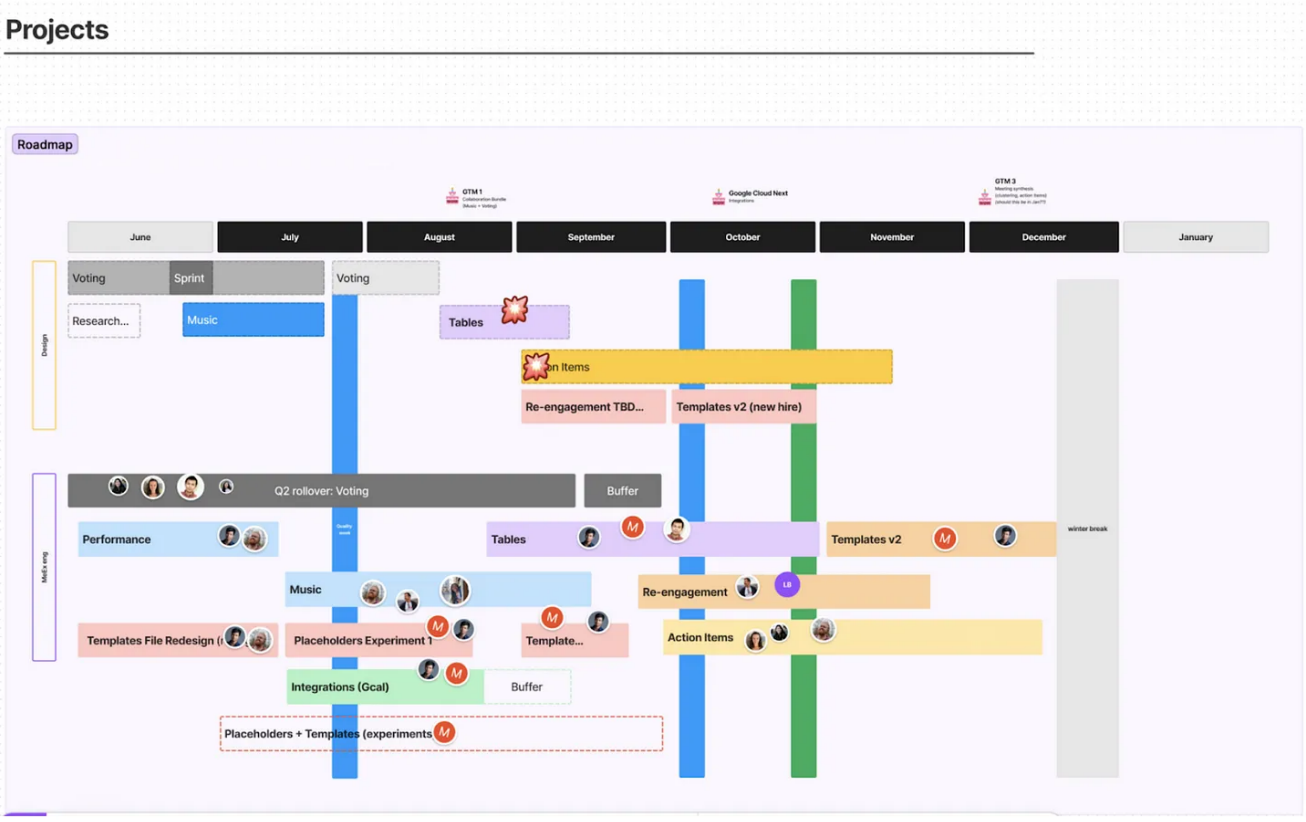
FigJam is ingrained in every engineering team. We use it to build engineering diagrams, to build architecture diagrams and share ideas, run meetings and to do roadmap planning. A few examples of how the engineering team uses FigJam:




Examples of FigJam use cases by the engineering team for calendar, roadmap, project updates, and cross-functional (XFN) updates




Engineering team project updates on FigJam




Roadmap visualization on FigJam

We use Figma Design widely within the company. Developers use it to inspect designs and figure out what they're supposed to implement. Figma Design has inspection capabilities to make it a little bit easier to get exact values for development when translating a design idea to production form.

Our engineers also have editor seats in Figma. And a lot of our customers do too, because they're often collaborating on designs with the designer and bringing their ideas and feedback into the Figma doc.

## Experimentation

When it comes to experimenting, Figma could be seen as in contrast to some stories you hear about social products. We don't subscribe to the extreme of 'everything is an experiment' or the opposite 'don't do any experiments' extreme: we aim for a balance.

We have certain launches that are very experiment-driven; for example, when we're trying to improve several aspects of the onboarding experience, or using a different checkout flow.

There's also plenty of cases when experimenting doesn't make sense. If we're going to release a new feature that introduces a new data type, then we need to make sure we have a *lot* of conviction before we give it to users. This is because if we ever take this data type away, it will feel like removing functionality.

**A lot of things still go through a more "binary" rollout, without the ability to be rolled back.** We're always looking for opportunities to experiment, but we recognize this isn't appropriate in some cases. At the same time, we use feature flags: everything gets turned on through feature flags and we have the ability to turn them off quickly if there's an issue.

We do a lot of upfront testing – private betas and alphas – to flush out potential issues as soon as possible. We aim to not have to roll things back after they've launched widely.

**We have dedicated environments in which it's easy to experiment, by design.** We have a beta release of our desktop application for people who opted in via our beta channel. Experimenting in the beta application is easy because the desktop application doesn't change the core data types of Figma. It's adding UI and features around it, so it's easier to try new things.

When it comes to a new feature launch or a new editing primitive, we approach it differently. We cannot rely on the beta app, so we always dogfood everything we do internally. We have our own staging environment, which is our internal Figma instance.

**We stress-test everything we build** to ensure we really are proud of it and love it, before putting it in front of customers. When we go to customers with an early alpha build, we might walk them through a developer cluster where they don't have their real data yet, and they just give high-level feedback. Before we launch some big bets, we try to get customers in front of the private beta release.

We work with our sales and developer advocate, who finds the right customers who wanted the new feature in the first place. We try to give them the ability to use it while it's still being developed, so we can iterate quickly based on their feedback.

## Quality weeks

We hold regular "quality weeks", to pay down quality debt, which can range from bugs, to limitations of features at launch. It's our way of ensuring we always reflect on things we launched previously and are making them better, versus always just looking ahead.

On top of quality weeks, we also make use of bug bashes, and share the progress of features with our customers, via early alphas.

## Onboarding engineers

How Figma onboards is pretty standard for a company of our size. Engineers start on a team and they also go through a series of onboarding presentations and exercises over the course of their first couple of months. There's a standard onboarding process for everyone at Figma where we walk them through our history and how different functions work together.

A big part of our onboarding goal for engineers is to help them ship something on their first day. We want to familiarize engineers with the way code flows from development to production. We also want to make them feel like they know how to deliver end-user impact as quickly as possible.

We balance:

- Contributions to the team: new joiners learning through impact.
- Meta-exercises: the bigger picture, how different systems work together, and getting new joiners comfortable working across our internal folder structures in the monorepo.

We have a monorepo at Figma and also different subdirectories, where some teams tend to spend more time. We want to ensure teams don't get overly confined to their area and feel confident contributing code throughout the code base, as needed.

### Internal mobility

We encourage people to switch teams as this kind of mobility is great. We've had a lot of people switch from infrastructure back to more product-focused teams, than the other way around. A unique thing about Figma is that you get to work across so many different domains on a relatively small code base and small engineering team. We think there's a lot of value in learning best practices across domains, and understanding shared commonalities and principles.

## 5. The engineering career ladder

In the early days of Figma, we were intentional about not having a career ladder, as a challenge of introducing a leveling system early is that you have to change it constantly. That's because what it means to be a senior engineer when a company is small, is different from what it means when the organization is larger, and engineers are working with more teams.

We told people we *did* have levels, but we wouldn't formalize the definition of each level because it would change. Our goal was for people to focus on this message:

> "Right now you should think about what's going to make your team and the company successful, and we're going to try to figure out the best way to measure the value you're creating, and compensate you and level you accordingly."

As we grew and things became more stable, we codified this statement. We ran a collaborative process across the engineering management group to collect leveling systems from peer companies. What did we like about what we had? What could we learn from other people and companies? We put together our first IC career ladder with that information and rolled it out in 2019, when we were at around 40 engineers.

**It's hard to build the perfect leveling system.** On one end, you don't want to be overly specific; people need space to be creative and figure out how to maximize their impact. At the other end, you also want to really coach people, guide them and give quality feedback. A good leveling system gives consistency to managers who are trying to provide quality feedback.

We rolled out an engineering management ladder after the IC ladder was in place. To create the engineering management ladder, we indexed from canonical management ladders you get in standardized compensation data sets, like many businesses do. Our goal was to provide a more generic view of what it means to be a manager, and then once we had a generalized ladder, we specialized it for engineering. We started to build the engineering manager career ladder at the beginning of 2022 and rolled it out at the end of the year.

Our levels go from 1 to 6, with 1 being entry-level engineering, and 6 being the highest level typically reserved for very key people with outsized impact.

**We were very intentional about not giving titles, as I think they're not always transferable between companies.** Different businesses use different titles and it's easy to over-index on the title and assume you have to be this or that title, in order to be comparable with other companies, but that's not always true.

While I definitely appreciate why people like to have titles, we try to find a better trade-off by being very specific about responsibilities. We don't publish levels, but we're not trying to hide them internally, either. Every engineer has the title 'software engineer'

internally. People can opt into sharing their level if they want, but we don't over-index on levels, as we don't want to discourage people from speaking up if they have a good idea because they have a lower level.

## 6. Tech stack

We use a monorepo for the majority of our code. We have a couple of dedicated repos for very special things or open source projects, but generally we try to encourage people to work in the same repo. It works pretty well in terms of encouraging code reuse and sharing.

Some of the technologies we use:

- Frontend:
  - React/Redux, TypeScript
  - WebAssembly, WebGL
  - Powering the canvas: C++, FreeType, HarfBuzz.
- Mobile: native iOS and native Android
- APIs: REST
- Backend:
  - Ruby, ActiveRecord
  - LiveGraph: a schema-based real-time database inspired by GraphQL. Built using a combination of TypeScript and Go.
  - Database: RDS with Postgres, AWS S3
- Infrastructure: running on top of AWS

We are building a culture where engineers aren't afraid to rethink how to architect an application in order to squeeze every last bit of performance and memory out of it. We encourage our engineers to not be limited by their specialization or abstraction boundaries. Instead, to leverage each other's collective knowledge to solve a hard problem when they face one. And when it comes to hard problems, don't just avoid them or come up with a work around; solve them!

# 7. Engineering challenges

Figma has engineering challenges that are pretty unique to us.

## Client-side performance

Our end users are professional designers who use this tool to get their job done. If we slow them down in any way, we're going to lose them, or we're just going to make them less effective. That's why performance is very, very important to us at Figma.

**High frames-per-second rendering** of dozens of screens, all at once. We are a product design tool where people are iterating on these ideas for their own products. They are often generating dozens or hundreds of variations of screens or web pages, and they pan, zoom and edit while doing it. We've built special-purpose renderers and data structures to maintain high frame rates even as we render complex pages.

At the same time, they are mostly static, except iteration designs for prototypes. In the early days of Figma, browser renderers were inconsistent and not always as performant. Luckily, this has changed for the better over time.

**Memory usage**. These screens are extremely data dense. Also, we cannot assume every user has access to every font, so we have to embed the resulting geometry and layout into our files. This means Figma files tend to be a lot larger than you'd assume, and certain kinds of edits require the entire file to be loaded. We solve this by incrementally loading files for viewers, and are ruthless about monitoring and managing memory, continuously looking for ways to minimize the size of various data structures.

**Latency**. Ensure editing works without waiting on a server round-trip. After all, we are competing with native applications which aren't always talking to the internet. We also need to create a sense of presence and collaboration when working with other people in the same file. This goes well beyond client-side: it's where we start to get into distributed systems problems.

## Distributed databases

Under the hood, when you think about a Figma file, it's easy to think of it as just a design file. But the reality is every Figma file is effectively a replica of a distributed database that can be accessed via a variety of devices and on our own servers. Challenges include:

- *Latency*: how do we ensure data changes are experienced by users in real time?
- *Scale*: how do we ensure the database is scalable?
- *Durability*: how do we ensure data is durable, so customers don't lose data if a server goes down?

*Concurrency* is another interesting area. We have to consider concurrency across all features to ensure everything is eventually consistent. For example:

- How do you prevent floating point oscillations from layout computation on different CPU architectures?
- How do you ensure components and instances always end up in a consistent state, even if two users try to pull in updates from a library file at the same time?
- How do you avoid recomputing text layout more than necessary, if two users are editing the same element at the same time?

We need to find the right balance between traditional database concerns like durability, and more novel goals like sub-millisecond real-time streaming. This involves trade-offs like whether or not to wait for a change to be checkpointed to disk, before sending it to other users connected to the same file at the same time.

We built our own database which optimizes for this design use case. We didn't write our own file storage engine, we used existing technologies to write changes to logs and checkpoint logs to snapshots. We have our own server endpoints which:

- Reconcile changes across clients
- Figure out which data are needed by different clients
- Figure out how to materialize partial views of the total data set, so users get exactly what they need, when they need it.

What about the CAP theorem? This theorem states any distributed data store can provide only two out of three guarantees:

- Consistency
- Availability
- Partition tolerance

Anyone who's looked into CAP theorem discovers that tradeoffs aren't as scary as they first seem. They result in important but relatively subtle trade-offs most people don't notice. In our case, we took a tradeoff on availability: if the application process on the server were to go down, clients are temporarily unable to load a file. These failures happen so rarely and we're now able to resolve them so quickly that it's unlikely you'll even notice.

In practice, we expect the window in which things could go wrong to be very small. Even if a server crashes, we expect customers to not lose much work. This is a tradeoff we have to make. The challenge for us and any other engineering team is to find the right set of trade-offs for our use case.

## Performance

Measuring performance is an important part of how we work. We measure the following:

- Using unit tests that stress actions and measure their duration in our CI pipelines, so we can catch regressions.
- Aggregate production percentiles, such as 50th or 90th percentile performance.
- We measure very specific use cases which we know have much stricter performance requirements. For example, when you're moving your mouse around a Figma file, you don't want that to feel slow or sluggish.

We go a level deeper than just measuring performance: we also think about distributions and the use cases of those distributions. Many companies only get so far as measuring average performance for all users. Even if you measure higher percentiles

like the 90th or 95th, you'll often still miss the challenges some of your most important users face.

**Percentile groups hide the power users.** There are certain users who might be pushing the limits of the product, but doing it in a way that a lot of other people – if they're successful – will do as well. We try to create segments of these customers and really focus on the challenges of these specific use cases.

How do we figure out what our user segments are? Surprisingly, a lot of this comes from talking to customers! For example, as an engineer, you might see a complaint on Twitter about some part of Figma being slow. You think "that's odd, that shouldn't be slow, it seems like a reasonable use case." So you actually reach out to that user and learn more about what they are doing.

Fortunately, at least with our community, users tend to be very open to getting help, so we'll often jump on a call with them. In some cases, we realize they are working with this big design team at a large company, or doing something we didn't anticipate with the product: and so now we need to be better. We then go back and figure out how we can show this valid use case in our data, and how we can build a segment of data which allows us to track progress in this area.

## Scale

Figma is extremely write-heavy. This is because designers are iterating on their own work more than they're viewing the work of others.

The challenge we have is to ensure we support a write-heavy distributed database, which behaves both as a real-time database, and is also extremely durable. One more constraint we have is having to preserve version history in a granular enough way. Some of our scaling challenges include:

- Spinning up new machines quickly and automatically recovering from failures
- Selecting partitioning and sharding strategies that minimize hardware exposure for a given customer, while ensuring seamless collaboration across teams and workspaces

- Helping customers detect, work around and resolve various networking issues, particularly around web sockets and firewalls

## Looking ahead

So far, we've optimized the product for designers. However, we are thinking about entire product teams, as we look ahead. How can we make Figma even more accessible and familiar to developers as well? Software engineers are a key audience of ours and if developers don't like Figma, then the collaboration loop can be broken.

We have plenty of room to improve how much developers working with designers "like" Figma, both from a general product ergonomics perspective, but also from a performance point of view.

# Takeaways

*Hi, this is Gergely, again.*

Thanks very much to Kris for sharing so much detail about how Figma works. Kris asked me to report that they are hiring, and have big ambitions for the product and growth.

As I talked with Kris, a few things stood out to me as being unique about Figma:

**A much more flexible leadership style.** To be honest, I was initially surprised by how unopinionated Kris was about some things. As CTO, he's decided to mandate very few things and to give the team a lot of freedom. But, an hour into our two-hour conversation, I start to feel that not mandating opinionated approaches was very purposeful by him.

It goes back to his professional background of working at a variety of startups, as an early employee at Asana and Airbnb, and realizing there's no one "best" approach, and that a motivated team does better than one that's told *how* to do things. On the other hand, Kris was unusually *opinionated* on a few things:

- **Intentional growth.** In common with Linear, Figma has also limited its pace of growth to not more than 100% in headcount, year-on-year. Kris is committed to not growing faster than this.
- **Milestones for projects.** Although Kris doesn't mandate much, milestones for every project is one thing he insists on. I agree with this approach, as discussed in the article Software engineers leading projects.
- **The push for quick feedback loops and collaboration.** Kris certainly means it when he talks about getting rapid feedback and collaborating cross-team. From the onboarding stage when engineers are pulled into cross-team activities, all the way to working on a single monorepo, Kris takes opportunities to make cross-team activities easier, and the design team certainly helps by inviting engineers to design crits.
- **A deliberately pragmatic experimentation approach.** I was surprised to hear Figma – and Kris in particular – are less set on the "everything is an experiment" approach which many product companies embrace. Instead, Kris is intent on being clear on what *is* and what *is not* a good candidate for experimentation. There is, of course, no one best approach, but not rushing into experiments seems to work fine for Figma.
- **Talk directly with customers.** I like how Kris leads by example in reaching out to customers on social media. It's eye-opening to learn that this approach helped the engineering team identify key customer segments, such as some "early power users." Without these reach outs, perhaps Figma would not have been able to adopt its product in a way that lets power users be more productive, and larger organizations to use the tool without performance issues.
- **Obsession with performance**. Figma – and Kris – obsess about performance far more than I've observed other product companies do. Looking at the adoption curve, this has definitely helped Figma.

**Taking pride in what you build.** If I was to take just one learning from my discussion with Kris, it's this:

> "We make sure we really are proud of [our work] and love it before we put it in front of any customers."

In many ways, Figma embodies the opposite of the "move fast and break things" mentality that dominated startups during the 2010s. It's refreshing to see this "build with deliberation, build with pride" approach has paid off.

There's no single best way to build a product or company. I hope you've enjoyed learning about the way that works for Figma, and perhaps get some inspiration for practices to experiment with at your team or company.

93 Likes · 2 Restacks

## 1 Comment

Write a comment...

**James Carr** Writes Bourbon and Barbells Apr 10

Great post... such a huge deep dive into Figma's Engineering culture. I am at the halfway point of their interview pipeline and have nothing but positive feedback on their process as well. So many companies have allowed their hiring process to fall by the wayside due to being overwhelmed by the number of applicants applying, it is refreshing to see a company provide timely feedback and stay in touch with an applicant!

LIKE (2) REPLY ···